

# How Amazon Q reduced the time Amazon developers spent waiting for technical answers by 450k hours this year

by Hank Cycyota and Alex Baluta | on 01 NOV 2024 | in [Amazon Q, Customer Solutions](#) | [Permalink](#) | [Share](#)

## Introduction

Software development is complex and time consuming. Developers frequently need to stop building to get answers to hard, technical questions. What is the error in my code? How do I debug the logic? Where do I go to find this information? In [2024 Stack Overflow Developer Survey](#) 53% of respondents agreed that waiting on answers disrupts their workflow, even if they know where to go find those answers. Similarly, 30% of respondents said knowledge silos impact their productivity ten times or more per week.

Our team – Amazon Software Builder Experience (ASBX) – leaned in to help solve this problem on behalf of Amazon developers. The ASBX organization has a mission to improve the software builder experience across tens of thousands of software engineers that work in all Amazon businesses. This includes the discovery tools that developers use to build and innovate on behalf of our customers. At Amazon, we have a wealth of software development expertise and an extensive knowledge base, but individual developers often have a hard time finding the exact information they need for the task at hand. They're looking for a needle in a haystack. We have a few internal tools where Amazon developers go to connect with those subject matter experts (SME) but for the most complex questions, they might need to wait hours before they get a response. However, often the information they need is buried somewhere deep in our knowledge base. We saw an opportunity to pair new techniques in generative AI like retrieval augmented generation (RAG) with our extensive knowledge base to reduce the demand on those SMEs in the tools that our developers use every day. In this way, we would reduce the time our developers had to spend waiting for answers, allowing them stay in their workflow and continue delighting customers.

While we thought RAG would enable us to better assist our developers, we knew that our solution would need to pass rigorous security and privacy bars and scale to support the volume of documentation and questions that Amazon developers generate. Amazon Q Business met those requirements as well as removed some of the duplicative work that comes with managing a separate index and large language model. Additionally, Amazon Q Business comes with some out-of-the-box APIs that would allow us to integrate it with the tools that Amazon developers use as part of their discovery workflows. Finally, those APIs come with important hooks that would let us use the context within those tools to improve information retrieval and answer relevancy.

This year, Amazon Q Business has helped tens of thousands of Amazon developers answer questions and get back to building. With Amazon Q Business connected to our internal knowledge repositories, our developers are getting unblocked quicker to deliver results for customers; we've been able to reduce the time it takes for Amazon developers to find answers to technical questions from hours to just a few seconds. This year, Amazon Q Business has resolved over 1 million internal Amazon developer questions, reducing time spent churning on manual technical investigations by more than 450k hours.

## Closing the discovery loop

Over its 30-year history, Amazon developers have generated a vast corpus of content to help them delight their customers. This includes community-generated content such as runbooks, dashboards, service-level documentation, structured Q&A, and program information. While we have an abundance of knowledge, finding the right answer to some of our more complex technical questions has been a challenge and the act of finding information pulls developers out of their workflow. When a developer is struggling to find an answer for a specific question, there are a few popular internal resources to get support from technical experts.

- Developers can post a question on our internal Q&A boards (a tool called Sage) and wait for a reply. These questions are often complex in nature and require specific expertise to answer. The challenge is, answers aren't immediate and the more obscure the question, the longer it can take for an SME to review and respond.
- Developers can find the appropriate interest channel in Slack and ask there (often channels that are set up by a team of experts to provide support for a given problem domain). These questions are similarly complex and a developer benefits from asynchronous back and forth with the SME. This route can be faster than our Q&A boards, but it can still take hours to get a reply.

In both cases, the developer needs to wait for an answer from those SMEs to unblock their task. They could transition to the next task on their sprint board but now they need to context switch to something new (and often quickly switch back once the SME finally responds). What's more, the answers to these questions often already exist somewhere in our knowledge base but the developer couldn't find that needle in the collective haystack of tools and repositories that we have at Amazon.

We saw an opportunity to better connect Amazon developers with answers to their questions by integrating Amazon Q Business with those tools they were already using. We wanted this solution to take on the role of a subject matter expert in tools like Sage and Slack to provide a fast answer to questions to get the developer unstuck. We ingested our internal knowledge repository consisting of millions of documents into Amazon Q Business so our developers could get answers based on data across those repositories. Then, we integrated our Amazon Q Business instance with the tools where our developers commonly ask questions. Finally, we used context inherent to the tools themselves (e.g., the Slack channel in which a user was asking a question or the specific Sage subject they were posting to) to provide more useful responses. This approach has resulted in three primary benefits:

- **Swift adoption:** We integrated the Q&A capabilities of Amazon Q Business into tools like Slack and Sage to make it part of the workflows our developers use every day. It also prevented us from having to create (yet) another tool that our developers needed to visit to get answers to questions. As a result, Amazon Q Business has already answered over 1 million internal Amazon developer questions this year.
- **More precise answers:** This approach has enabled better responses to developer questions via Amazon Q Business by using the context that is readily available on the tool. In this way, we can narrow the retrieval scope from potentially millions of documents. Developers report that answers are more helpful when we scope retrieval in this manner (versus when no scope is present).
- **Faster answers:** Leveraging Amazon Q Business has reduced the time it takes for a developer to get an answer to seconds, getting them unblocked so they can delight customers.

The remainder of this article touches briefly on how we set up our Amazon Q Business instance and how we integrated it with downstream tools. We then go into more depth around how we leveraged tool-specific context to provide better answers in service of getting our developers unstuck faster.

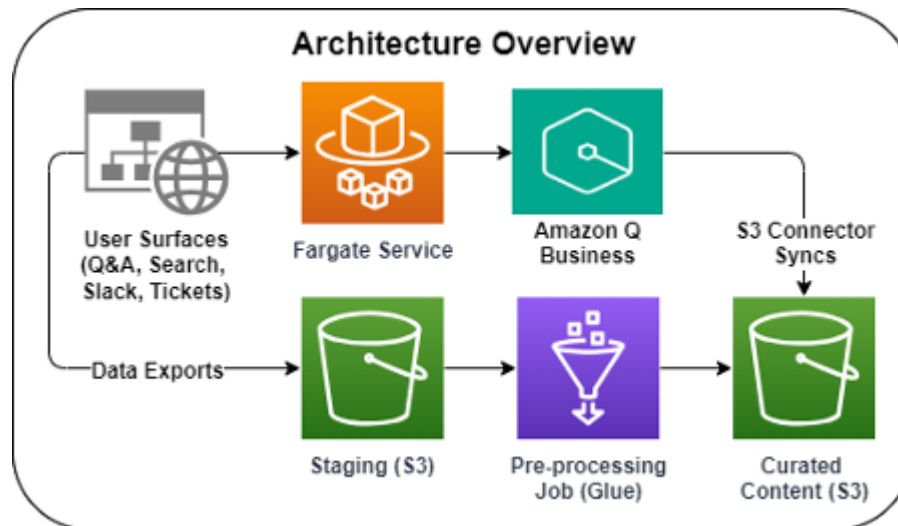
## Setting up Amazon Q Business: Integrating Q Business with our knowledge repositories

We took a straightforward approach to setting up our Amazon Q Business application and followed the steps outlined in the service documentation [here](#). We staged our knowledge repositories in S3 and used the Amazon Q Business S3 Connector to ingest those documents into our index (more info on how to use those S3 connectors [here](#)). We have a lot of content that isn't useful in retrieval – pre-processing of our documents in S3 to remove stale content allows us to reduce our repository of over ten million potentially useful documents to under four million relevant ones. We also stage in S3 to enrich content with metadata (e.g., document type, team or service name, URL hierarchy) that isn't in the source repository in order to take advantage of more Q features (e.g., filtering, boosting).

## Integrating Amazon Q Business: Putting Q Business where users are

We leveraged a single Amazon Q Business application that hosts all our curated content to quickly release our desired experience in places where our developers ask questions. Also, leveraging one Amazon Q Business application lets us provide a consistent experience on the tools we integrate with, ensuring users are getting the same answers from the same dataset.

As we considered the UX for these integrations, our goal was that these experiences complement the existing workflow in the tools developers use. For instance, rather than increasing the cognitive load on the developer by adding a chat experience in the corner of Sage, we chose to automatically respond with answers to questions on the Q&A board. Similarly, developers can invoke Amazon Q Business on Slack in an interest channel to get answers to questions. We learned early that users have expectations around what sort of questions a bot is able to answer based on where they are interacting with it. For instance, customers were frustrated at a prototype experience on Sage that didn't actually include the Q&A data native to that tool. Our prerequisite for onboarding new tools is to make sure we have the data for that tool and have considered other expectations users might have.



## Enhancing our solution: Improving retrieval through surface-level context

By integrating Amazon Q Business directly with applications that developers use every day we've been able to leverage that information as context to improve retrieval accuracy and give better responses to developers. A common technical question our Amazon Q Business application answers is "How do I onboard to XX service?" Depending on the service in question (e.g., How common is the service name? Have many other actors onboarded to it?), Amazon Q Business might retrieve context outside of the authoritative set of documentation we want it to use. This can be confusing to the reader and reduce trust in the answers. However, if we know that a developer is asking that question in the XX-service@ Slack channel, we can narrow the retrieval scope to just the content related to XX-service.

As an example of where this helps, there is an internal Amazon service framework called Coral. Take a common question like "How do I maintain my Coral dashboard?" The answer to this question can vary by team, most of which maintain specific documentation about their dashboards. Asking this question without scoping will list a lot of good best practices around dashboards. However, if we know that the question is being asked in a specific team channel on Slack, it can use that context to scope to that specific team's documentation to provide a more precise answer.

0:00



We scope documents with the Amazon Q Business filter attribute, which allows us to customize and control chat responses based on the document metadata reflected in index fields. We can break this down to the following steps:

- We capture and associate document-specific metadata during ingestion.
- We enable SMEs to define a topic space based on that metadata. A topic space is a collection of authoritative documents (spread across multiple repositories) specific to their knowledge domain. To use an example for an external-facing repository, a topic space could encompass all the Dynamo content on AWS Docs as specified by a root URL (like <https://docs.aws.amazon.com/dynamodb>). In this case, we would include all the URLs under that root in that topic space.
- With the topic space defined, we leverage the Amazon Q Business metadata filters to restrict its RAG down at runtime to just the content in the topic space. Then, when a user asks a question about that topic space, say in a Slack interest channel on DynamoDB, the filters are applied and a domain-specific answer is generated from the SME-specified, authoritative content. Our call looks like this and the service documentation to set up your own is [here](#).
- In cases where the topic space does not contain an answer to the developer's question (maybe the documentation hasn't been updated or the question is too specific), we also added an opt-in user configuration to enable falling back to general knowledge. This ignores the filters and retries the original query to take advantage of the whole dataset.

## Conclusion

This post described our process to integrating Amazon Q Business into tools where internal Amazon developers are looking for support. It also describes how we used context from these tools to improve the responses we provide to customers by using the inbuilt Amazon Q Business filter attributes.

You can also leverage this approach to make it easier for your developers to find answers to questions. For more information on how to do this, along with other ways you can make Amazon Q work for you, visit [Getting started with Amazon Q Business](#).